

Laboratorio 1 — Inferencia de una red neuronal con NumPy

Clase: IAR401 — Redes Neuronales y Aprendizaje Profundo

Periodo: Q3 2026

Catedrático: Ing. Iván de Jesús Deras Tábor

Propósito

En este laboratorio implementará, **desde cero y usando NumPy**, el proceso de inferencia de una red neuronal multicapa preentrenada para clasificar imágenes de dígitos manuscritos del conjunto MNIST.

El objetivo no es entrenar una red ni utilizar un framework de aprendizaje profundo. El objetivo es comprender y programar el recorrido hacia adelante (*forward propagation*) de una red ya entrenada: cargar sus parámetros, ejecutar sus capas, obtener probabilidades y medir su desempeño.

Este laboratorio suma a la calificación del **proyecto de la clase**. Además, se realizará una evaluación en papel para comprobar individualmente que cada estudiante alcanzó los objetivos del laboratorio.

Objetivos de aprendizaje

Al completar el laboratorio, el estudiante será capaz de:

1. Cargar un modelo serializado en JSON en un diccionario de Python.
2. Interpretar la arquitectura, los pesos y los sesgos de una red neuronal densa.
3. Cargar datos almacenados en formato NumPy .npz.
4. Implementar una red neuronal para inferencia mediante clases propias y NumPy.
5. Implementar las funciones de activación **ReLU** y **Softmax**.
6. Aplicar preprocesamiento, aplanamiento (*flattening*) y propagación hacia adelante respetando las dimensiones de los tensores.
7. Generar predicciones de clase y calcular la exactitud (*accuracy*) sobre el conjunto de prueba.
8. Explicar las decisiones principales de su implementación y sus resultados.

Archivos iniciales

Los archivos requeridos se encuentran en Starter-Files/:

Archivo	Descripción
mnist_mlp.json	Modelo preentrenado: arquitectura, pesos y sesgos.
mnist_test.npz	Conjunto de prueba MNIST: imágenes y etiquetas reales.

Use rutas relativas para que su programa pueda ejecutarse desde la carpeta del laboratorio.

Restricciones y herramientas permitidas

- Puede usar durante el desarrollo cualquier herramienta que considere útil: Jupyter Notebook, Google Colab, VS Code, un archivo `.py` ejecutado desde la terminal, depuradores, documentación, visualizaciones, etc.
- La implementación que se demuestre debe realizar la inferencia con **clases propias y NumPy**.
- No se permite delegar la construcción o la inferencia de la red a Keras, TensorFlow, PyTorch, scikit-learn u otro framework de aprendizaje profundo.
- No debe reentrenar el modelo ni alterar los pesos o sesgos provistos.
- No se entrega código fuente directamente en la plataforma. El código debe publicarse en un repositorio de GitHub accesible para el docente, como evidencia de lo mostrado en el video.

Descripción del modelo

El archivo `mnist_mlp.json` contiene un diccionario con tres campos principales:

- `input_shape`: forma original de cada imagen: `[28, 28]`.
- `preprocess.scale`: valor de normalización: `255.0`.
- `layers`: lista ordenada de capas densas y sus parámetros.

La arquitectura esperada es:

Capa	Operación	Pesos	Sesgos	Activación
Entrada	Imagen aplanada	—	—	—
1	Densa	(784, 128)	(128,)	ReLU
2	Densa	(128, 10)	(10,)	Softmax

Para un lote de m ejemplos, la salida de una capa densa se calcula como:

$$Z = A W + b$$

Donde:

- A es la entrada de la capa, de forma (m, n_{entrada}) .
- W es la matriz de pesos, de forma $(n_{\text{entrada}}, n_{\text{salida}})$.
- b es el vector de sesgos, de forma $(n_{\text{salida}},)$; NumPy lo replica por *broadcasting* para cada ejemplo del lote.
- Z tiene forma (m, n_{salida}) .

Las activaciones requeridas son:

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

Para evitar problemas numéricos, implemente Softmax de forma estable: reste el máximo de cada fila antes de aplicar la exponencial.

Actividades requeridas

1. Cargar e inspeccionar el modelo

1. Abra `mnist_mlp.json` con el módulo estándar `json` y cárguelo en un diccionario de Python.
2. Recorra el campo `layers` e identifique, para cada capa:
 - tipo de capa;
 - número de unidades;
 - función de activación;
 - forma de `w`;
 - forma de `b`.
3. Convierta los pesos y sesgos provenientes del JSON a arreglos de NumPy antes de utilizarlos en los cálculos.

2. Implementar la red neuronal

Implemente una solución orientada a objetos. Como mínimo, su diseño debe incluir:

- una clase para una capa densa, que conserve `w`, `b` y el nombre de su activación;
- una clase `NeuralNetwork` (o nombre equivalente) que contenga las capas en orden;
- un método para ejecutar la propagación hacia adelante de un lote de ejemplos;
- funciones propias para `relu` y `softmax`.

La red debe construirse a partir de la configuración leída del archivo JSON; no codifique manualmente los valores de pesos, sesgos ni la arquitectura.

3. Cargar y preparar el conjunto de prueba

1. Cargue `mnist_test.npz` con `np.load`.
2. Identifique los arreglos `images` y `labels`.
3. Verifique las dimensiones esperadas:
 - `images`: (10000, 28, 28);
 - `labels`: (10000,).
4. Convierta las imágenes a punto flotante y normalícelas usando el valor `preprocess.scale` del modelo:

$$X_{normalizado} = \frac{X}{255.0}$$

5. Aplane cada imagen de (28, 28) a un vector de 784 características. El lote completo debe tener forma (10000, 784).

4. Ejecutar inferencia y evaluar

1. Ejecute la propagación hacia adelante sobre el conjunto completo de prueba.
2. Compruebe que la salida final tenga forma (10000, 10).
3. Verifique que los valores de cada fila de la salida Softmax sumen aproximadamente 1.
4. Obtenga la clase predicha mediante el índice de mayor probabilidad.
5. Calcule la exactitud:

$$accuracy = \frac{1}{m} \sum_{i=1}^m 1(\hat{y}_i = y_i)$$

6. Muestre al menos una imagen del conjunto, su etiqueta real, la clase predicha y, además, las probabilidades de las 10 clases.

Puntos de verificación

Antes de grabar el video, valide como mínimo lo siguiente:

- ☐ El JSON fue cargado como un diccionario y el modelo fue creado a partir de dicho diccionario.
- ☐ Las dimensiones de pesos y sesgos coinciden con la arquitectura indicada.
- ☐ La entrada se normaliza y se aplane a (n_ejemplos, 784).
- ☐ ReLU y Softmax fueron implementadas con NumPy.
- ☐ Softmax es numéricamente estable y las probabilidades por ejemplo suman aproximadamente 1.
- ☐ La red realiza inferencia sobre las 10,000 imágenes de prueba.
- ☐ Se calculó y mostró la exactitud del modelo.
- ☐ Se mostró al menos una predicción individual con su imagen y etiqueta real.

Como referencia técnica, el modelo provisto alcanza aproximadamente **96.66 %** de exactitud al aplicar correctamente el preprocesamiento y la inferencia.

Video de demostración individual

Cada estudiante deberá subir un video individual de **máximo 5 minutos** y proporcionar el enlace a su repositorio de GitHub.

Reglas obligatorias para el video

1. El video debe ser un **screencast**. Se recomienda usar **OBS Studio** para grabar la pantalla y la cámara web; puede utilizar otra herramienta equivalente.
2. El estudiante debe aparecer en cámara durante **todo el video**, por ejemplo, mediante una ventana de cámara superpuesta al screencast. Su rostro debe ser visible.

3. Debe explicar la arquitectura de su implementación: las clases definidas, cómo se construyen las capas a partir del JSON y las partes más importantes de la propagación hacia adelante.
4. Debe mostrar una corrida real de su código que incluya la carga de los archivos, una predicción individual y el cálculo de la precisión sobre todo el conjunto de prueba.
5. El código mostrado en el video debe estar disponible en un repositorio de **GitHub** accesible para el docente. El repositorio permitirá validar la implementación presentada. No se debe entregar código fuente directamente en la plataforma.

El video debe mostrar claramente:

1. La estructura general de su implementación con clases propias.
2. La carga del JSON y la construcción de la red a partir de las capas definidas en el archivo.
3. La implementación o explicación de ReLU y Softmax.
4. La carga, normalización y aplanamiento de los datos de prueba.
5. La ejecución de una predicción individual: imagen, etiqueta real, clase predicha y probabilidades o confianza.
6. La evaluación de todo el conjunto de prueba y la exactitud obtenida.
7. La respuesta a esta pregunta conceptual:

¿Por qué la capa final usa Softmax y qué significa que las probabilidades de sus 10 salidas sumen aproximadamente 1?

El video debe permitir verificar que el estudiante comprende y puede explicar su propia implementación. Durante la evaluación posterior se podrán realizar preguntas adicionales para comprobar que los objetivos del laboratorio fueron alcanzados individualmente.

Ponderación del laboratorio

La calificación total de este laboratorio se distribuye de la siguiente manera:

Componente	Ponderación
Video de demostración y código publicado en GitHub	40 %
Evaluación manuscrita	60 %
Total	100 %

Rúbrica del componente video y código

Criterio	Evidencia esperada	Ponderación dentro del 40 %
Construcción del modelo desde JSON	Carga correcta del diccionario y creación de	20 %

Criterio	Evidencia esperada	Ponderación dentro del 40 %
Implementación con NumPy	capas con los parámetros provistos. Clases propias, capa densa, propagación hacia adelante, ReLU y Softmax.	25 %
Preparación correcta de datos	Carga de .npz, normalización por 255 y aplanamiento a 784 características.	15 %
Predicción y medición	Corrida visible, predicción individual, evaluación completa y cálculo correcto de exactitud.	20 %
Explicación, presencia y repositorio	Estudiante visible durante todo el screencast, explicación clara, respuesta conceptual y repositorio verificable.	20 %
Total		100 %

Recomendaciones

- Trabaje primero con una o pocas imágenes para depurar las dimensiones y, luego, evalúe el conjunto completo.
- Imprima las formas de los arreglos en cada etapa; la mayoría de los errores provienen de dimensiones incompatibles.
- No aplique Softmax elemento a elemento ni sobre todo el lote a la vez: debe normalizarse por cada ejemplo.
- Use operaciones vectorizadas de NumPy; evite procesar las 10 000 imágenes una por una cuando no sea necesario.
- Mantenga una separación clara entre: carga del modelo, preprocesamiento, definición de clases, inferencia y evaluación.

Preguntas de reflexión

1. ¿Qué error produciría omitir el aplanamiento de las imágenes antes de la primera capa densa?
2. ¿Por qué es necesario normalizar los píxeles de $[0, 255]$ a $[0, 1]$ en este modelo?
3. ¿Qué representa cada posición de la salida de tamaño 10?
4. ¿Por qué `argmax` permite convertir las probabilidades de Softmax en una clase predicha?